

Yapılar (Structures)

Aralarında mantıksal bir ilişki bulunan farklı türden bilgiler "yapılar (structures)" içerisinde mantıksal bir bütün olarak ifade edilebilir. Yapılar diziler gibi bellekte sürekli kalır. Bir yapı içerisindeki elemanlara üye (member) denir. Üyelerin herbiri farklı veri tipine sahip olabilir. Bu sayede, kendi tipinizi üretebilirsiniz.

Genel yapı bildirimi:

```
struct yapı_adı{
    tip yapı_değişken_ismi;
    tip yapı_değişken_ismi;
    ...
};
```

Bir öğrenciye ait bilgileri bir çatı altında aşağıdaki gibi toplanabilir:

```
/* kayıt veri tipi! */
struct kayıt{
    /* üyeler */
    char ad[10];
    long no;
    short sınıf;
};
```

Bu tipte bir değişken tanımlama:

```
struct kayıt ogr1, ogr2;
```

şeklinde olabilir. ogr1 değişkeni ile tanımlanan 1. öğrencinin numarasına bir değer atama işlemi:

```
ogr1.no = 2012597;
```

şeklinde yapılır. Bu deyimimin kullanımı Program 1 ve 2’de gösterilmiştir.

Program 1: struct deyiminin kullanımı

```
01: /* Bir öğrenciye ait bilgilerin struct deyimi ile bir çatı altında
02: toplanması */
03:
04: #include <stdio.h>
05:
06: /* kayıt yapısı */
07: struct kayıt{
08:     char ad[10];
09:     long no;
10:     int sınıf;
11: };
12:
13: int main()
14: {
15:     struct kayıt ogr; /* ogr değişkeni kayıt tipinde */
```

```

16:
17:     printf("Ogrenci Nosu   : "); scanf("%ld",&ogr.no);
18:     printf("Ogrenci Adi    : "); scanf("%s" , ogr.ad);
19:     printf("Ogrenci Sinifi: "); scanf("%d" ,&ogr.sinif);
20:
21:     printf("\n*** Girilen bilgiler ***");
22:     printf("\nNo          : %ld",ogr.no);
23:     printf("\nAdi         : %s ",ogr.ad);
24:     printf("\nSinifi      : %d ",ogr.sinif);
25:
26:     return 0;
27: }

```

ÇIKTI
Ogrenci Nosu : 948589
Ogrenci Adi : Ahmet
Ogrenci Sinifi: 2

*** Girilen bilgiler ***
No : 948589
Adi : Ahmet
Sinifi : 2

Yapılar, diğer değişkenler gibi, fonksiyonlara parametre olarak geçirelilebilir.

Program 2: struct deyiminin kullanımı

```

01: /* Yapıların bir fonksiyona parametere olarak aktarılabilmesi */
02:
03: #include <stdio.h>
04:
05: struct TARİH{
06:     int gun,ay,yil;
07: };
08:
09: void goster(struct TARİH x){
10:     printf("Tarih: %02d-%02d-%4d\n", x.gun, x.ay, x.yil);
11: }
12:
13: int main()
14: {
15:     struct TARİH n; /* n değişkeni TARİH tipinde */
16:
17:     n.gun = 1;
18:     n.ay  = 8;
19:     n.yil = 2003;
20:
21:     goster(n);
22:
23:     return 0;
24: }
25:

```

ÇIKTI
Tarih: 01-08-2003

Yapılarla da, gösterici tanımlamaları yapılabilir. Ancak, bir yapı gösteren göstericinin, gösterdiği yere yönlendirme operatörü, ->, ile erişilir.

Program 3: struct deyiminin kullanımı

```
01:  /* 15prg05.c: Bir yapı gösteren gösterici */
02:
03:  #include <stdio.h>
04:  #include <stdlib.h>
05:
06:  struct Meyve{
07:      float agirlik;
08:      float fiyat;
09:  };
10:
11:  int main()
12:  {
13:      struct Meyve *muz, elma;
14:      float muzTutar, elmaTutar;
15:
16:      /* muz Meyve tipinde bir gösterici */
17:      muz = (struct Meyve*) malloc( sizeof(struct Meyve) );
18:      muz->agirlik = 2.50;
19:      muz->fiyat   = 3.50;
20:      muzTutar = muz->fiyat * muz->agirlik;
21:
22:      /* elma Meyve tipinde bir deęisken */
23:      elma.agirlik = 2.00;
24:      elma.fiyat   = 1.75;
25:      elmaTutar = elma.fiyat * elma.agirlik;
26:
27:      printf("Meyve      Agirlik  Birim Fiyatı      TUTAR (TL)\n");
28:      printf("-----      - - - - -      - - - - -      - - - - -\n");
29:
30:      printf("Muz          %7.2f          %7.2f          %7.2f\n",
31:             muz->agirlik, muz->fiyat, muzTutar);
32:
33:      printf("Elma          %7.2f          %7.2f          %7.2f\n",
34:             elma.agirlik, elma.fiyat, elmaTutar);
35:
36:      return 0;
37:  }
```

ÇIKTI

Meyve	Agirlik	Birim Fiyatı	TUTAR (TL)
-----	-----	-----	-----
Muz	2.50	3.50	8.75
Elma	2.00	1.75	3.50

typedef Deyimi

struct ile oluşturulan yapıda typedef deyimi kullanılırsa, bu yapıdan deęişken tanımlamak için tekrar struct deyiminin kullanılmasına gerek kalmaz.

```
typedef struct kayit{
    char  ad[10];
    long  no;
```

```
    short sinif;  
} ogr1,ogr2;
```

Program 2 küçük bir değişikliğe *Program 4* yeniden yazılmıştır.

Program 4: *typedef - struct deyiminin kullanımı*

```
01: /* Yapıların bir fonksiyona parametere olarak  
02: aktarılabilmesi.  
03:     typedef deyimi kullanıldığında struct deyimine  
04: gerek yoktur */  
05:  
06: #include <stdio.h>  
07:  
08: typedef struct{  
09:     int gun,ay,yil;  
10: }TARİH;  
11:  
12: void goster(TARİH x){  
13:     printf("Tarih: %02d-%02d-%4d\n", x.gun, x.ay,  
14: x.yil);  
15: }  
16:  
17: int main(void)  
18: {  
19:     TARİH n;  
20:  
21:     n.gun = 1;  
22:     n.ay  = 8;  
23:     n.yil = 2003;  
24:  
25:     goster(n);  
26:  
    return 0;  
}
```

ÇIKTI

Tarih: 01-08-2003

typedef başka kullanımı da vardır. C dilinde program kodları bu deyimle tamamen türkçeleştirilebilir. Örneğin bu deyim:

```
typedef int tamsayi;
```

şeklinde kullanılırsa programda daha sonra int tipinde bir değişken tanımlarken şu biçimde kullanılmasına izin verilir.

```
tamsayi x,y;    /* int x,y anlamında */
```

Bağlı Listeye veri ekleme işlemi:

Linked listlere yeni bir düğüm ekleme işlemini 3 farklı yere yapabiliriz. Listenin en başına, listenin en sonuna, ve herhangi iki düğüm arasına.

Şöyle bir yapı oluşturduğumuzu düşünelim;

```
typedef struct
linked_list {
    int data;
    struct linked_list
*next;
} NODE;
```

Bu şekilde bir yapıya art arda yeni düğümler eklemek için aşağıdaki kod parçacığını kullanabiliriz.

```
void insert(int value){

    if(head==NULL){ /// ilk düğüm oluşturulmadıysa önce onu
    oluşturuyoruz.
        head = (NODE *)malloc(sizeof(NODE));
        head->data = value;
        head->next = NULL;
        tail = head; /// Sadece ilk düğüm için, head = tail
    } else {
        NODE *temp = (NODE *)malloc(sizeof(NODE)); ///
Eklenecek düğümler için memory allocation
        temp->data = value;
        temp->next = NULL;
        tail->next = temp;
        tail = tail->next;
    }
    printf("Eklenen data %d\n", value);
}
```